

Library Reservation App

React application developed with GitHub Copilot for managing library book reservations.

GitHub Repository:

<https://github.com/imaaaan05/software-engineering-pipeline/tree/library-app>

Project Description

This project consists of a small library reservation system developed in React using Visual Studio Code and GitHub Copilot.

The application allows users to:

- Register with an email
- View available books
- Reserve and cancel reservations
- Add new books dynamically
- Persist data using localStorage
- Use a responsive and modern interface

The development process was iterative and based on prompts provided to GitHub Copilot.

Technologies Used

- React
 - Vite
 - JavaScript
 - CSS
 - GitHub Copilot
 - localStorage
-

Features

User Registration

- Email validation
- Success message after registration
- User data saved in localStorage

Book Reservation System

- Reserve books
- Cancel reservations
- Reservation status updates in real time

Add New Books

- Dynamic form to add books
- Title and author inputs
- Automatic rendering in the book list

Data Persistence

- Books remain saved after refreshing the page
- User information stored locally

Responsive UI

- Modern card layout
 - Styled buttons and forms
 - Responsive design for different screen sizes
-

Development Workflow with GitHub Copilot

The application was developed progressively using an iterative workflow with GitHub Copilot.

Step 1 — Initial Book List

Prompt: Create a React component that displays a list of books with reserve buttons.

At this stage, the application displayed a static list of books with simple reserve buttons.

Step 2 — Reservation Logic

Prompt: Add a reservation toggle system using React useState.

The reservation system was improved by adding dynamic state management with React useState, allowing users to reserve and cancel reservations.

Step 3 — Component Refactoring

Prompt: Create reusable BookCard and BookList components.

The codebase was reorganized into reusable React components to improve scalability and maintainability.

Step 4 — Add New Books

Prompt: Create a React form component for adding books with title and author inputs.

A form was added so users could dynamically create new books directly from the interface.

Step 5 — Data Persistence

Prompt: Add localStorage persistence so books remain after refreshing the page.

Persistence was implemented using localStorage so the application state remained saved after refreshing the browser.

Step 6 — User Registration

Prompt: Create a user registration form with email input, validation, success message and localStorage persistence.

A registration system was implemented with email validation and persistent storage for user data.

Step 7 — UI Improvements

Prompt: Improve the UI styling of the library app with modern responsive CSS cards, buttons and form styles.

The interface was redesigned with a more modern responsive layout using improved CSS styling generated and refined with Copilot.

Step 8 — Search System

Prompt: Add a search bar to filter books by title in real time.

A real-time search system was added to improve usability and user experience. **---

Screenshots

Main Application

Library Reservation App

Reserve books, manage your collection, and keep data saved locally.

Search by title

User Registration

Email

Register

Add a New Book

Title

Author

Add Book

Harry Potter

Joanne Rowling

Reserve

Clean Code

Robert C. Martin

Reserve

The Pragmatic Programmer

Andrew Hunt

Reserve

Design Patterns

Gang of Four

Reserve

Registration Form

User Registration

Email

eduard.herrera01@estudiant.upf.edu

Register

Registration successful! Logged in as eduard.herrera01@estudiant.upf.edu.

Reservation System

Harry Potter Joanne Rowling Cancel Reserved	Clean Code Robert C. Martin Cancel Reserved	The Pragmatic Programmer Andrew Hunt Reserve	Design Patterns Gang of Four Reserve
--	--	--	--

Reflection About GitHub Copilot

GitHub Copilot helped accelerate the development process considerably, especially during the creation of React components, forms, CSS styling and localStorage persistence.

The workflow was highly iterative. Instead of generating the complete application in a single step, the project evolved progressively through increasingly specific prompts. Each iteration refined the previous implementation and added new functionality.

One of the main advantages of Copilot was the speed at which repetitive code and UI structures could be generated. It was especially useful for:

- Creating reusable React components
- Managing React state with useState and useEffect
- Generating responsive CSS layouts
- Implementing forms and validation logic
- Adding localStorage persistence

However, the generated code was not always correct on the first attempt. Several problems appeared during development:

- Some generated code required manual fixes
- Component imports occasionally failed
- Styling needed adjustments after generation
- Certain prompts generated incomplete implementations
- The application structure had to be reorganized manually

Because of this, manual review and debugging were essential throughout the project. GitHub Copilot worked best as an assistant rather than a fully autonomous developer.

Overall, the experience was very positive. The combination of iterative prompting, manual corrections and progressive refinement allowed the application to evolve efficiently while maintaining control over the final result.

How to Run the Project

Install dependencies:

```
npm install
```

Run the development server:

```
npm run dev
```

Open in browser:

```
http://localhost:5173
```